STACKFORGEAI

StackFix

Product Development Standard Operating Procedure

Full Lifecycle Guide: Design → Build → Launch → Scale

Document	StackFix SOP v1.0
Product	StackFix Repair Management System
Owner	StackForgeAI
Market	Kigali, Rwanda (East Africa)
Version	1.0 — May 2026
Status	Active — Internal Reference

Table of Contents

Executive Summary & Product Vision

StackFix is a repair-shop management SaaS purpose-built for Rwanda. This SOP is the definitive reference for every engineer, designer, and product manager working on the platform.

What We Are Building

StackFix is a cloud-based, multi-tenant SaaS platform that enables repair businesses (phone workshops, electronics centres, IT service providers, authorised retailers) in Kigali and across Rwanda to run every aspect of their operation from one dashboard: job intake, invoicing, USSD payment collection, automated customer notifications via WhatsApp and SMS, technician workflow management, and business analytics.

Why It Matters

- Rwanda's repair economy is large and growing — most shops still run on paper tickets and manual payment collection.
- Mobile money (MoMo) is ubiquitous; USSD works on every phone with no internet needed by the customer.
- WhatsApp penetration is very high in Kigali — automated status updates dramatically reduce follow-up calls.
- No direct competitor offers a locally-priced, locally-supported, Kinyarwanda-capable repair management platform.

Strategic Goals

- Launch a production-grade web app with full technician + admin dashboards.
- Integrate MTN Mobile Money USSD payments natively.
- Deliver automated WhatsApp Business API and SMS notifications.
- Ship a companion mobile app (React Native) for technicians in the field.
- Maintain a clean RESTful API so third-party and mobile clients can consume all features.
- Be deployable in 72 hours for a new repair shop, zero technical knowledge required from the operator.

Development Phases at a Glance

Phase	Focus Area	Target Completion
1	Design System & Brand	Week 1
2	UI/UX Design & Prototyping	Weeks 2–3
3	Dev Infrastructure & Tooling	Week 3
4	Database Architecture	Week 4
5	Backend Development	Weeks 4–7
6	Frontend Development	Weeks 5–9
7	API Design & Mobile Endpoints	Weeks 7–9
8	MTN Mobile Money Integration	Weeks 8–10
9	WhatsApp & SMS Notifications	Weeks 9–10
10	Mobile App Development	Weeks 10–14
11	Testing & QA	Weeks 13–15
12	Hosting, DevOps & Infrastructure	Weeks 14–15
13	Launch Strategy	Week 16
14	Post-Launch Operations	Ongoing

PHASE 1

Design System & Brand

The design system is the single source of truth for every visual decision made on the platform. Everything — web app, mobile app, emails, invoices — must derive from it.

1.1 Brand Foundation

Colour Palette

The brand is already established on the landing page. Codify it formally:

Token	Hex Value	Usage	Notes
--color-primary	#00B341	CTAs, active states, badges	StackFix green
--color-primary-dark	#007A2E	Hover states, headings	
--color-bg-dark	#0D1F12	Hero sections, sidebar	Dark mode base
--color-bg-light	#F9FAFB	App background	
--color-surface	#FFFFFF	Cards, panels	
--color-text-primary	#111827	Body copy	
--color-text-secondary	#6B7280	Labels, meta	
--color-border	#E5E7EB	Dividers, inputs	
--color-error	#EF4444	Error states	
--color-warning	#F59E0B	Warnings, pending	
--color-success	#10B981	Success, paid	

Typography Scale

Token	Size	Weight	Usage

--text-xs	12px / 0.75rem	400	Labels, timestamps
--text-sm	14px / 0.875rem	400–500	Table data, meta
--text-base	16px / 1rem	400	Body copy
--text-lg	18px / 1.125rem	500–600	Card headings
--text-xl	20px / 1.25rem	600	Section titles
--text-2xl	24px / 1.5rem	700	Page headings
--text-3xl	30px / 1.875rem	700	Dashboard KPIs
--text-4xl	36px+	800	Landing hero

Typefaces: Inter (primary UI), JetBrains Mono (code/ticket IDs). Load via Google Fonts CDN. Store font tokens in design system config.

Spacing System

Use a base-4 spacing scale. Define as CSS custom properties:

- --space-1: 4px | --space-2: 8px | --space-3: 12px | --space-4: 16px
- --space-6: 24px | --space-8: 32px | --space-12: 48px | --space-16: 64px

Elevation & Shadows

Three levels of elevation for cards, modals, and dropdowns. Document shadow tokens formally so they are consistent across web and mobile.

1.2 Component Library

Atomic Components (must be designed and coded before any screen work)

All components must be built as reusable units in the component library before any screen is assembled.

Category	Components	States Required	Notes
Inputs	Text input, Textarea, Select, Search, Date picker	Default, Focus, Error, Disabled	RW phone number format validation
Buttons	Primary, Secondary, Ghost, Icon, Loading	Default, Hover, Active, Disabled, Loading	Green primary CTA always

Badges	Status badge (Pending/Under Repair/Completed/Picked Up/Paid)	5 status states	Colour-coded by status
Cards	Ticket card, Invoice card, Stat card, KPI card	Default, Hover, Selected	
Tables	Data table, Sortable table, Mobile-responsive table	Empty state, Loading skeleton	
Modals	Confirmation, Form modal, Alert modal	Open, Close animations	
Navigation	Sidebar, Top nav, Breadcrumbs, Mobile bottom nav	Active, Hover, Collapsed	
Notifications	Toast (success/error/warning/info), Alert banner	Auto-dismiss, Persistent	
Forms	Ticket form, Invoice form, Settings form	Validation states	AI suggest integration
Data Viz	Revenue chart, Status donut, KPI trend line	Loading, Empty, Error	Recharts library

1.3 Design Tokens File Structure

Store all tokens in a shared config consumed by both the web app and mobile app:

- tokens/colors.ts — all colour tokens
- tokens/typography.ts — font families, sizes, weights, line heights
- tokens/spacing.ts — spacing scale
- tokens/shadows.ts — elevation tokens
- tokens/breakpoints.ts — responsive breakpoints (sm: 640, md: 768, lg: 1024, xl: 1280)

1.4 Status System (Critical for StackFix)

Status colours must be consistent across every surface — dashboard, mobile, SMS message content, and invoice PDFs:

Status	Colour	Hex
Pending	Amber/Orange	#F59E0B
Under Repair	Blue	#3B82F6
Completed	Green	#10B981
Picked Up	Purple	#8B5CF6
Paid	Dark Green	#00B341
Cancelled	Red	#EF4444

1.5 Invoice & PDF Brand Template

The invoice is a customer-facing document. It must be professionally branded:

- Business logo top-left, StackFix badge top-right
- Customer name, phone, device, fault — clearly structured
- Line items table: Part | Quantity | Unit Price | Total
- Subtotal, VAT (18%), Grand Total highlighted in brand green
- USSD payment code in a prominent box with instructions in English and Kinyarwanda
- Footer: business address, phone, email, RDB registration number

Generate invoices as PDF using Puppeteer (server-side HTML-to-PDF). Store the HTML template in the design system.

PHASE 2

UI/UX Design & Prototyping

Design is already underway on Lovable (stackfix.lovable.app). This phase formalises the design process, screen inventory, and handoff standards.

2.1 Design Tool Stack

Tool	Purpose	Notes
Lovable.app	Current design-to-code environment	Already in use — primary output is React code
Figma	Design specs, component library documentation, annotations	Designers share specs from here
FigJam	User flow diagrams, stakeholder workshops	
Storybook	Component documentation and visual testing	Deployed alongside app
Chromatic	Visual regression testing for UI components	Integrated with Storybook

2.2 Screen Inventory — Web App

Authentication

- Login page (Admin + Technician — single entry point, role-based redirect)
- Forgot password flow (email reset)
- First-time setup wizard (business profile, add first technician)

Admin Dashboard

- Overview dashboard: KPI cards (total jobs, revenue, pending, today's completions)
- Revenue chart (weekly/monthly toggle), Status breakdown donut chart
- Recent activity feed
- Quick actions: New Ticket, View Invoices, Export

Ticket Management

- All tickets — filterable table (by status, technician, date range, device type)

- Ticket detail page — full lifecycle view, notes, status history, invoice link
- New ticket form — customer info, device, fault, AI-assist suggestions inline
- Edit ticket — update parts, notes, status

Invoice Management

- Invoice list — filter by status (draft/sent/paid/overdue), date, amount
- Invoice detail — full invoice preview, PDF download button, send via WhatsApp/SMS
- Create/edit invoice — line item builder, VAT auto-calculation, USSD code generator

Payment History

- Payment log — all confirmed payments, filterable by date range and technician
- Export controls — CSV and PDF download
- Outstanding balance view — unpaid invoices

Team Management (Admin Only)

- Technician list — add, edit, deactivate accounts
- Role and permission assignment
- Activity log per technician

Settings

- Business profile — name, address, logo, RDB number
- Payment model toggle — Pay Before Service / Pay on Pickup
- Language toggle — English / Kinyarwanda (full app translation)
- Notification templates — customise WhatsApp and SMS message content
- USSD configuration — link to MTN MoMo business account
- VAT settings — toggle and rate configuration

2.3 Screen Inventory — Technician View

Technicians see a simplified interface — only what they need:

- My tickets — active jobs only, sorted by urgency
- New ticket form — same as admin but scoped to technician
- Ticket detail — update status, add notes, view invoice
- Quick stats — personal job count, completions today

2.4 Responsive Design Rules

Breakpoint	Layout	Notes
Mobile < 640px	Single column, bottom nav	Full feature parity for technicians
Tablet 640–1024px	Two column, collapsible sidebar	
Desktop > 1024px	Three column, full sidebar expanded	Primary admin experience

2.5 Accessibility Standards

- WCAG 2.1 AA minimum compliance
- All interactive elements keyboard navigable
- Colour contrast ratio ≥ 4.5:1 for all text on backgrounds
- Screen reader labels on all icon-only buttons
- Form error messages tied to fields via aria-describedby

2.6 Design Handoff Process

1. Designer marks screens as 'Ready for Dev' in Figma
2. Developer receives Figma link + component spec
3. Lovable.app used to scaffold React code from design
4. Developer refines Lovable output, wires up real data
5. QA compares implementation against Figma — reports visual regression in Chromatic
6. Designer signs off before feature is merged

PHASE 3

Development Infrastructure & Tooling

A well-configured development environment prevents technical debt and enables the team to ship confidently and quickly.

3.1 Repository & Version Control

Item	Choice	Rationale
Platform	GitHub	Best CI/CD integration, team familiarity
Structure	Monorepo (Turborepo)	Web app, mobile app, API, shared packages in one repo
Branching	Git Flow (main / develop / feature/* / hotfix/*)	Clear release management
Commit style	Conventional Commits (feat: / fix: / chore: etc)	Enables auto-changelog
PR policy	All code reviewed by 1+ engineer before merge	Quality gate
Protected branches	main and develop require PR + passing CI	No direct pushes

Monorepo Structure

- apps/web — Next.js frontend
- apps/mobile — React Native app (Expo)
- apps/api — Node.js/Express REST API
- packages/ui — Shared component library
- packages/tokens — Design tokens
- packages/types — Shared TypeScript types
- packages/utils — Shared utility functions
- packages/config — Shared ESLint, TypeScript, Tailwind configs

3.2 Technology Stack

Frontend — Web App

Technology	Version	Purpose	Notes
Next.js	14+	React framework, SSR/SSG, routing	App Router — use server components
TypeScript	5+	Type safety across all code	Strict mode enabled
Tailwind CSS	3+	Utility-first styling	Configured with design tokens
shadcn/ui	Latest	Accessible base component library	Customised to StackFix brand
Zustand	4+	Client-side state management	Lightweight — replaces Redux
TanStack Query	5+	Server state, caching, mutations	Used for all API calls
React Hook Form	7+	Form management	With Zod validation
Zod	3+	Schema validation	Shared types with API
Recharts	2+	Data visualisation (charts)	Revenue, status charts
date-fns	3+	Date formatting/manipulation	
i18next	23+	Internationalisation (EN/RW)	Kinyarwanda translation
Lucide React	Latest	Icon library	

Backend — API

Technology	Version	Purpose	Notes
Node.js	20 LTS	Runtime	LTS for stability
Express.js	4+	HTTP framework	Fast, minimal, well-understood
TypeScript	5+	Type safety	Shared types with frontend

Prisma ORM	5+	Database access layer	Type-safe queries, migrations
PostgreSQL	15+	Primary relational database	Supabase-hosted
Redis	7+	Caching, session store, job queues	Upstash (serverless Redis)
BullMQ	4+	Background job processing	Notification delivery, exports
Zod	3+	Input validation on all endpoints	Shared with frontend
JWT	—	Authentication tokens	Access + refresh token pattern
bcrypt	—	Password hashing	12 rounds minimum
Helmet	—	HTTP security headers	
express-rate-limit	—	API rate limiting	Prevent abuse
Morgan	—	HTTP request logging	
Winston	—	Structured application logging	JSON logs to Logtail

3.3 Local Development Setup

Prerequisites

- Node.js 20 LTS — via nvm (node version manager)
- pnpm 9+ — package manager (faster than npm, works perfectly with monorepos)
- Docker Desktop — local PostgreSQL and Redis containers
- Git — version control
- VS Code — recommended IDE with defined workspace extensions

Recommended VS Code Extensions

- ESLint, Prettier — code quality
- Prisma — schema highlighting and format
- Tailwind CSS IntelliSense — class autocomplete
- Thunder Client or REST Client — API testing

- GitLens — enhanced Git history
- Error Lens — inline error display

Environment Variables

Every service has its own .env file. Never commit .env to Git. Use .env.example as the template. Required variables:

- DATABASE_URL — PostgreSQL connection string
- REDIS_URL — Redis connection string
- JWT_SECRET — minimum 64 character random string
- JWT_REFRESH_SECRET — separate secret for refresh tokens
- MTN_MOMO_API_KEY — MTN MoMo collection API key
- MTN_MOMO_SUBSCRIPTION_KEY — Ocp-Apim-Subscription-Key header value
- MTN_MOMO_TARGET_ENVIRONMENT — sandbox or production
- MTN_MOMO_CALLBACK_URL — webhook URL for payment notifications
- WHATSAPP_API_TOKEN — Meta WhatsApp Business API token
- WHATSAPP_PHONE_NUMBER_ID — WhatsApp Business phone number ID
- SMS_API_KEY — Africa's Talking API key
- SMS_SENDER_ID — registered sender ID (e.g. STACKFIX)
- NEXT_PUBLIC_API_URL — public-facing API base URL
- NEXT_PUBLIC_APP_URL — public-facing web app URL
- PUPPETEER_EXECUTABLE_PATH — for PDF generation on server

3.4 Code Quality Standards

Tool	Config	Purpose
ESLint	eslint-config-next + custom rules	Catch bugs and enforce patterns
Prettier	Shared config in packages/config	Consistent code formatting
Husky	Pre-commit hooks	Run lint + type-check before every commit
lint-staged	Only lint changed files	Performance in large repos
TypeScript strict	strict: true in tsconfig	Maximum type safety
Knip	Remove unused code/exports	Keep codebase clean

3.5 CI/CD Pipeline

Use GitHub Actions for all automation:

On every Pull Request

7. Install dependencies (pnpm install)
8. Type check (pnpm tsc --noEmit)
9. Lint (pnpm lint)
10. Unit + integration tests (pnpm test)
11. Build check (pnpm build)
12. Chromatic visual regression (if UI changes detected)

On merge to develop

13. All PR checks pass
14. Deploy to staging environment automatically
15. Run E2E tests against staging
16. Notify team in Slack/WhatsApp

On merge to main

17. All PR checks pass
18. Deploy to production (with approval gate)
19. Run smoke tests against production
20. Tag release with version number
21. Update changelog

PHASE 4

Database Architecture

The database is the foundation of StackFix. Every business-critical piece of data — tickets, invoices, payments, customers — lives here. It must be designed for correctness, performance, and extensibility.

4.1 Database Technology Choice

Primary Database: PostgreSQL 15+ hosted on Supabase.

- Supabase provides managed PostgreSQL with built-in auth, Row Level Security (RLS), realtime subscriptions, and storage
- Supabase's free tier covers development; paid tier covers production with automatic backups
- Access via Prisma ORM — type-safe queries, migration system, and schema management

Cache Layer: Redis via Upstash (serverless). Used for:

- Session storage (JWT refresh token storage)
- Rate limiting counters
- Short-term caching of dashboard KPIs (invalidated on write)
- BullMQ job queues for background tasks

4.2 Core Schema Design

Multi-Tenancy Model

StackFix is multi-tenant — each repair shop is a separate Organisation. Every table that holds shop-specific data has an organisation_id foreign key. Row Level Security policies in PostgreSQL enforce that queries only return data belonging to the requesting organisation.

Entity Relationship Overview

Core entities and their relationships:

- Organisation (1) → (many) Users
- Organisation (1) → (many) Customers
- Organisation (1) → (many) Tickets
- Ticket (1) → (1) Invoice
- Invoice (1) → (many) InvoiceLineItems
- Invoice (1) → (many) Payments

- Ticket (1) → (many) StatusHistory
- Ticket (1) → (many) TicketNotes
- Ticket (1) → (1) Device (embedded or FK)

Table Specifications

organisations

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	
name	VARCHAR(255)	NOT NULL	Shop trading name
slug	VARCHAR(100)	UNIQUE, NOT NULL	URL identifier e.g. 'ace-repairs'
address	TEXT		
phone	VARCHAR(20)		
email	VARCHAR(255)		
logo_url	TEXT		Supabase Storage URL
rdb_number	VARCHAR(50)		Rwanda Development Board registration
payment_model	ENUM	NOT NULL, DEFAULT 'pay_on_pickup'	pay_before / pay_on_pickup
default_vat_rate	DECIMAL(5,2)	DEFAULT 18.00	
language	ENUM	DEFAULT 'en'	en / rw
momo_account_id	VARCHAR(255)		MTN MoMo linked account
plan	ENUM	DEFAULT 'starter'	starter / growth / enterprise
is_active	BOOLEAN	DEFAULT true	
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

users

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations.id	
full_name	VARCHAR(255)	NOT NULL	
email	VARCHAR(255)	UNIQUE, NOT NULL	Used for login
phone	VARCHAR(20)		
password_hash	TEXT	NOT NULL	bcrypt hashed
role	ENUM	NOT NULL	admin / technician
is_active	BOOLEAN	DEFAULT true	Soft deactivation
last_login_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

customers

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations, RLS	
full_name	VARCHAR(255)	NOT NULL	
phone	VARCHAR(20)	NOT NULL	Primary contact — WhatsApp/SMS
email	VARCHAR(255)		Optional
total_repairs	INTEGER	DEFAULT 0	Computed/cached
created_at	TIMESTAMPTZ	DEFAULT NOW()	

tickets

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations, RLS	
ticket_number	VARCHAR(20)	UNIQUE per org	e.g. TKT-0042
customer_id	UUID	FK → customers	
technician_id	UUID	FK → users	Assigned technician
device_type	VARCHAR(100)	NOT NULL	Phone / Laptop / Tablet etc
device_brand	VARCHAR(100)		Samsung / Apple / Tecno etc
device_model	VARCHAR(100)		Galaxy A54 / iPhone 14 etc
fault_description	TEXT	NOT NULL	Customer-reported issue
internal_notes	TEXT		Technician notes (not shown to customer)
status	ENUM	NOT NULL, DEFAULT 'pending'	pending/under_repair/completed/picked_up/cancelled
priority	ENUM	DEFAULT 'normal'	low/normal/high/urgent
estimated_completion	DATE		Optional ETA
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

invoices

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations, RLS	

ticket_id	UUID	FK → tickets, UNIQUE	One invoice per ticket
invoice_number	VARCHAR(20)	UNIQUE per org	e.g. INV-0042
customer_id	UUID	FK → customers	Denormalised for performance
subtotal	DECIMAL(12,2)	NOT NULL	Sum of line items
vat_rate	DECIMAL(5,2)	DEFAULT 18.00	
vat_amount	DECIMAL(12,2)	NOT NULL	Computed
total_amount	DECIMAL(12,2)	NOT NULL	subtotal + vat
currency	CHAR(3)	DEFAULT 'RWF'	Rwandan Franc
status	ENUM	DEFAULT 'draft'	draft/sent/paid/partially_paid/overdue/cancelled
ussd_code	VARCHAR(100)		Generated USSD payment string
ussd_reference	VARCHAR(100)		MoMo transaction reference for lookup
due_date	DATE		
paid_at	TIMESTAMPTZ		When payment confirmed
pdf_url	TEXT		Supabase Storage URL of generated PDF
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

invoice_line_items

Column	Type	Constraints	Notes
id	UUID	PK	
invoice_id	UUID	FK → invoices ON DELETE CASCADE	

description	TEXT	NOT NULL	e.g. 'Screen Replacement'
quantity	DECIMAL(8,2)	DEFAULT 1	
unit_price	DECIMAL(12,2)	NOT NULL	
total_price	DECIMAL(12,2)	NOT NULL	quantity × unit_price
item_type	ENUM	DEFAULT 'labour'	part/labour/diagnostic/other
sort_order	INTEGER	DEFAULT 0	Display order

payments

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations, RLS	
invoice_id	UUID	FK → invoices	
amount	DECIMAL(12,2)	NOT NULL	
currency	CHAR(3)	DEFAULT 'RWF'	
payment_method	ENUM	NOT NULL	momo_ussd / cash / card / other
momo_transaction_id	VARCHAR(255)		MTN MoMo reference ID
momo_payer_msisdn	VARCHAR(20)		Customer phone who paid
status	ENUM	DEFAULT 'pending'	pending/successful/failed/reversed
confirmed_at	TIMESTAMPTZ		When MoMo confirmed
notes	TEXT		
created_at	TIMESTAMPTZ	DEFAULT NOW()	

ticket_status_history

Column	Type	Constraints	Notes
id	UUID	PK	
ticket_id	UUID	FK → tickets	
from_status	ENUM		Previous status
to_status	ENUM	NOT NULL	New status
changed_by_user_id	UUID	FK → users	Who changed it
trigger	ENUM		manual/system/payment_confirmation
created_at	TIMESTAMPTZ	DEFAULT NOW()	

notifications

Column	Type	Constraints	Notes
id	UUID	PK	
organisation_id	UUID	FK → organisations	
ticket_id	UUID	FK → tickets	
customer_id	UUID	FK → customers	
channel	ENUM	NOT NULL	whatsapp / sms
message_body	TEXT	NOT NULL	Actual message sent
status	ENUM	DEFAULT 'pending'	pending/sent/delivered/failed
provider_message_id	VARCHAR(255)		WhatsApp/AT message ID
sent_at	TIMESTAMPTZ		
delivered_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	DEFAULT NOW()	

4.3 Database Indexes

Critical indexes to create for query performance:

- tickets: (organisation_id, status) — dashboard filter queries
- tickets: (organisation_id, created_at DESC) — default list ordering
- tickets: (customer_id) — customer history lookup
- invoices: (organisation_id, status) — invoice list filter
- payments: (organisation_id, confirmed_at DESC) — payment history
- customers: (organisation_id, phone) — phone number lookup
- ticket_status_history: (ticket_id) — status timeline

4.4 Database Migration Strategy

- All schema changes managed via Prisma Migrate
- Migrations committed to Git alongside code changes
- Never run raw SQL migrations manually in production
- CI/CD pipeline runs prisma migrate deploy automatically on deploy
- Shadow database used in development for migration testing
- Backups: Supabase Point-in-Time Recovery (PITR) enabled from day one

PHASE 5

Backend Development

The backend is the engine of StackFix — it processes all business logic, enforces security, communicates with payment providers, and delivers notifications.

5.1 Architecture Pattern

Layered architecture inside the Express API:

- Routes layer — HTTP route definitions and middleware attachment
- Controller layer — parse request, call service, return response
- Service layer — all business logic lives here (no DB calls)
- Repository layer — all database queries via Prisma
- Middleware layer — auth, validation, rate limiting, error handling
- Jobs layer — BullMQ background workers for async tasks

5.2 Authentication System

Strategy: JWT Access + Refresh Token

22. User submits email + password to POST /auth/login
23. Server verifies password via bcrypt.compare
24. Server issues: Access Token (15 min TTL) + Refresh Token (30 days TTL)
25. Access token stored in memory (JavaScript variable) on frontend
26. Refresh token stored in httpOnly, Secure, SameSite=Strict cookie
27. Every API request sends Authorization: Bearer <access_token>
28. When access token expires, frontend auto-calls POST /auth/refresh
29. Refresh token validated against Redis — invalidated on logout

Role-Based Access Control (RBAC)

- Admin: full access to all routes in their organisation
- Technician: access only to their own tickets and actions within their organisation
- Middleware enforces org isolation — no cross-tenant data access possible

5.3 API Route Groups

Group	Base Path	Notes

Authentication	/api/v1/auth	Login, refresh, logout, password reset
Organisation	/api/v1/org	Profile, settings, billing, language
Users / Team	/api/v1/users	CRUD for team members, role management
Customers	/api/v1/customers	CRUD, search, repair history
Tickets	/api/v1/tickets	Full ticket lifecycle
Invoices	/api/v1/invoices	Create, update, send, PDF generate
Payments	/api/v1/payments	Payment history, MoMo webhook receiver
Notifications	/api/v1/notifications	Resend, logs, template management
Analytics	/api/v1/analytics	Dashboard KPIs, revenue stats
AI	/api/v1/ai	Device suggest, fault suggest, invoice pre-fill
Export	/api/v1/export	CSV and PDF export endpoints
Webhooks	/api/v1/webhooks	MTN MoMo callbacks, WhatsApp status hooks

5.4 Business Logic Rules

Ticket Number Generation

Ticket numbers are per-organisation sequential: TKT-0001, TKT-0002. Use PostgreSQL sequences scoped per org. Never rely on application-level counters — use a DB function to generate the next number atomically.

Status Transition Rules

Enforce valid status transitions server-side — never trust the client:

- pending → under_repair (allowed), completed (not allowed), picked_up (not allowed)
- under_repair → completed (allowed), pending (allowed for rollback)

- completed → picked_up (allowed)
- picked_up → [terminal state — no further transitions]
- Any status → cancelled (admin only)

Every status change writes a row to ticket_status_history automatically (database trigger or service-level hook).

Invoice Rules

- Only one invoice per ticket — enforced at DB level with UNIQUE constraint
- Invoice total is always: sum(line_items) + VAT — computed by the server, never trusted from client
- An invoice can only be marked Paid when a confirmed payment record exists for the full amount
- USSD code is generated and stored on invoice creation — format: *182*8*1*AMOUNT*REFERENCE#

5.5 AI Integration

StackFix's AI assist features are powered by calling an LLM API (OpenAI GPT-4o or Claude via Anthropic API) from the backend. The AI logic is:

Device + Fault Auto-Suggest

When a technician types partial device text, the frontend calls GET /api/v1/ai/suggest-device?q={partial}. The backend queries a pre-built device model database first (fast, no LLM cost). Falls back to LLM for unusual models.

Invoice Pre-fill

When creating an invoice, the frontend calls POST /api/v1/ai/suggest-invoice-items with { device_type, device_model, fault_description }. The LLM returns suggested line items as JSON. The technician reviews and edits before saving.

AI Cost Control

- Cache AI responses by (device_model + fault_type) key in Redis — TTL 7 days
- Rate limit AI endpoints per organisation per hour
- Log all AI API calls for cost monitoring

5.6 PDF Invoice Generation

Server-side PDF generation using Puppeteer (headless Chrome):

30. API receives POST /api/v1/invoices/{id}/generate-pdf
31. Server renders the invoice HTML template with invoice data injected
32. Puppeteer prints the rendered HTML to PDF
33. PDF uploaded to Supabase Storage (private bucket, signed URLs)
34. pdf_url saved to invoice record
35. Signed URL returned to client for download or sharing

5.7 Background Jobs (BullMQ)

Queue	Jobs	Notes
notifications	send-whatsapp, send-sms, retry-failed	Retry 3x with exponential backoff
payments	poll-momo-status, reconcile-payments	Poll MoMo API for unconfirmed payments
exports	generate-csv, generate-pdf-report	Heavy jobs off main thread
cleanup	purge-old-logs, archive-old-tickets	Scheduled via cron

5.8 Error Handling Standard

All API errors follow a consistent JSON structure. Never expose internal errors to the client:

- { success: false, error: { code: 'TICKET_NOT_FOUND', message: 'Ticket not found', statusCode: 404 } }
- HTTP 400 — validation errors (include field-level detail)
- HTTP 401 — unauthenticated
- HTTP 403 — unauthorised (authenticated but insufficient permission)
- HTTP 404 — resource not found
- HTTP 409 — conflict (e.g. invoice already exists for ticket)
- HTTP 422 — business rule violation (invalid status transition)
- HTTP 429 — rate limit exceeded
- HTTP 500 — internal server error (log internally, generic message to client)

PHASE 6

Frontend Development

The web frontend is the primary interface for admins and technicians. It must be fast, reliable, and easy to use for non-technical repair shop operators.

6.1 Next.js App Structure

Using Next.js 14 App Router with the following directory structure:

- app/(auth)/login — Login page (unauthenticated route group)
- app/(auth)/setup — First-time onboarding wizard
- app/(dashboard)/layout.tsx — Shared dashboard layout with sidebar and header
- app/(dashboard)/page.tsx — Main overview dashboard
- app/(dashboard)/tickets — Ticket list and detail pages
- app/(dashboard)/invoices — Invoice management
- app/(dashboard)/payments — Payment history
- app/(dashboard)/team — Team management (admin only)
- app/(dashboard)/settings — Organisation settings
- app/(dashboard)/analytics — Reports and charts
- app/api — Next.js route handlers (thin proxy to Express API for SSR data)

6.2 State Management Strategy

Server State — TanStack Query

All data that comes from the API is server state. Use TanStack Query for:

- Fetching and caching ticket lists, invoice data, dashboard stats
- Optimistic updates — status changes reflect immediately before server confirmation
- Background refetch — dashboard auto-refreshes every 60 seconds
- Infinite scroll for long ticket lists

Client State — Zustand

Only use Zustand for UI state that doesn't come from the server:

- Currently selected filters (date range, status filter)
- Sidebar open/closed state
- Notification toast queue
- New ticket form draft (unsaved state)

6.3 Real-Time Updates

Supabase Realtime provides WebSocket-based live updates for ticket status changes:

36. Frontend subscribes to the tickets table for the current organisation
37. When a ticket status changes (e.g. payment confirmed → under_repair), all open browser tabs update automatically
38. Dashboard KPI cards re-fetch automatically
39. Technician receives a toast notification if their assigned ticket changes

6.4 Internationalisation (EN / Kinyarwanda)

Full UI translation using i18next and react-i18next:

- All user-visible strings use t('key') — no hardcoded English in JSX
- Translation files: locales/en/common.json and locales/rw/common.json
- Language toggle in settings persists to user profile in DB
- Date formatting respects locale (Kinyarwanda uses same numerals but different labels)
- Currency always displays as RWF with Rwandan Franc formatting

6.5 Performance Standards

- Lighthouse score targets: Performance ≥ 90, Accessibility ≥ 95, Best Practices ≥ 95, SEO ≥ 90
- Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1
- All images use next/image for automatic optimisation and lazy loading
- Heavy components (charts, PDF viewer) are dynamically imported
- API responses cached at CDN edge where appropriate
- Dashboard initial load target: < 1.5s on a 4G mobile connection in Kigali

6.6 Offline Capability (Progressive Enhancement)

Connection quality in Rwanda can be intermittent. The app must degrade gracefully:

- Next.js Service Worker caches the app shell
- Last-fetched ticket data readable offline (TanStack Query cache persisted to IndexedDB)
- New ticket forms show an 'offline' indicator — save draft locally, sync when back online
- Payment confirmations always wait for server acknowledgement

6.7 Security on the Frontend

- Never store JWT access tokens in localStorage — memory only
- Refresh tokens in httpOnly cookies — not accessible to JavaScript
- All API calls through a central axios/fetch instance with interceptors
- Content Security Policy headers configured in next.config.js
- Input sanitisation on all user-generated content before display
- Role-based UI rendering — admin-only routes redirect if technician accesses them

PHASE 7

API Design & Mobile Endpoints

The REST API is the single source of truth for all StackFix clients — the web app, the mobile app, and any future third-party integrations. It must be designed to be consumed easily by all these clients.

7.1 API Design Principles

- RESTful conventions — resources as nouns, HTTP verbs for actions
- Versioned from day one: /api/v1/ — allows non-breaking future evolution
- All responses are JSON with a consistent envelope format
- Pagination on all list endpoints: cursor-based pagination (not offset — better for realtime data)
- All timestamps in ISO 8601 UTC format
- All monetary amounts as integers in the minor currency unit (RWF has no minor unit — use whole numbers)
- All IDs as UUIDs — never expose sequential integers externally

7.2 Response Envelope Format

Every API response uses this shape:

- Success list: { success: true, data: [...], pagination: { cursor, hasMore, total } }
- Success single: { success: true, data: { ...resource } }
- Error: { success: false, error: { code, message, details? } }

7.3 Complete Endpoint Specification

Authentication Endpoints

Method + Path	Auth Required	Description	Notes
POST /api/v1/auth/login	No	Login with email + password	Returns access token + sets refresh cookie
POST /api/v1/auth/refresh	Refresh cookie	Issue new access token	

POST /api/v1/auth/logout	Access token	Invalidate refresh token	
POST /api/v1/auth/forgot-password	No	Send password reset email	
POST /api/v1/auth/reset-password	Reset token	Set new password	
GET /api/v1/auth/me	Access token	Get current user profile	

Ticket Endpoints

Method + Path	Auth Required	Description	Notes
GET /api/v1/tickets	Admin/Tech	List all tickets with filters	?status=&technician_id=&date_from=&date_to=&cursor=
POST /api/v1/tickets	Admin/Tech	Create new repair ticket	Triggers customer notification
GET /api/v1/tickets/:id	Admin/Tech	Get single ticket with full detail	Includes status history, invoice, notes
PATCH /api/v1/tickets/:id	Admin/Tech	Update ticket fields	Partial update
PATCH /api/v1/tickets/:id/status	Admin/Tech	Change ticket status	Validates transition rules
POST /api/v1/tickets/:id/notes	Admin/Tech	Add a note to ticket	
DELETE /api/v1/tickets/:id	Admin only	Soft-delete (cancel) ticket	

Invoice Endpoints

Method + Path	Auth Required	Description	Notes
GET /api/v1/invoices	Admin/Tech	List invoices with filters	?status=&date_from=&date_to=
POST /api/v1/invoices	Admin/Tech	Create invoice for a ticket	One per ticket enforced
GET /api/v1/invoices/:id	Admin/Tech	Get invoice with line items	
PATCH /api/v1/invoices/:id	Admin/Tech	Update invoice (if not paid)	
POST /api/v1/invoices/:id/send	Admin/Tech	Send invoice to customer	WhatsApp + SMS
POST /api/v1/invoices/:id/generate-pdf	Admin/Tech	Generate PDF and return URL	
POST /api/v1/invoices/:id/mark-paid	Admin only	Manually mark as paid (cash)	Creates payment record

Customer Endpoints

Method + Path	Auth Required	Description	Notes
GET /api/v1/customers	Admin/Tech	List customers with search	?q=name/phone
POST /api/v1/customers	Admin/Tech	Create new customer	Phone required

GET /api/v1/customers/:id	Admin/Tech	Customer profile + repair history	
PATCH /api/v1/customers/:id	Admin/Tech	Update customer info	

Analytics Endpoints

Method + Path	Auth Required	Description	Notes
GET /api/v1/analytics/dashboard	Admin	KPI summary cards	?period=today/week/month/custom
GET /api/v1/analytics/revenue	Admin	Revenue chart data	?period=&group_by=day/week/month
GET /api/v1/analytics/technician-performance	Admin	Per-technician stats	
GET /api/v1/analytics/device-breakdown	Admin	Most common devices and faults	

Webhook Endpoints (public — no auth, signature verified)

Method + Path	Purpose	Security	Notes
POST /api/v1/webhooks/momo-payment	MTN MoMo payment callback	X-Callback-Signature header verified	Updates payment + ticket status
POST /api/v1/webhooks/whatsapp-status	WhatsApp message delivery status	X-Hub-Signature-256 verified	Updates notification delivery status

7.4 API Documentation

- All endpoints documented in OpenAPI 3.0 spec (openapi.yaml in repo root)
- Swagger UI hosted at /api/docs in development and staging environments
- Postman collection exported from OpenAPI and committed to repo
- Every endpoint change requires OpenAPI spec update in the same PR

7.5 API Security

- All endpoints over HTTPS only — HTTP redirects to HTTPS
- CORS configured to allow only known origins (web app URL, mobile app)
- Rate limiting: 100 req/min per IP for unauthenticated, 1000 req/min per organisation for authenticated
- All incoming request bodies validated with Zod schemas before processing
- SQL injection impossible via Prisma parameterised queries
- Webhook payloads verified via HMAC signature — reject if signature fails

PHASE 8

MTN Mobile Money Integration (Rwanda)

MTN Mobile Money (MoMo) is Rwanda's dominant payment method. This integration is the single most critical technical piece of StackFix — it must be bulletproof.

8.1 How MoMo Works for StackFix

StackFix uses MTN MoMo's Collections API. The flow:

40. Technician creates invoice and it auto-generates a USSD code for the customer
41. Customer dials USSD code on their phone (*182*8*1*AMOUNT*REFERENCE# for MTN, or equivalent)
42. Customer confirms payment with their MoMo PIN
43. MTN sends a payment notification to StackFix's webhook URL
44. StackFix verifies the payment via the Collections API
45. Invoice status updates to Paid automatically
46. Ticket status updates to Under Repair (Pay Before model) or Picked Up (Pay on Pickup)
47. Customer and technician both receive confirmation notifications

8.2 MTN MoMo API Setup

Step 1: Register as an MoMo Partner

- Apply to MTN Rwanda Business for a Mobile Money for Business (MoMo for Business) account
- Register on the MTN MoMo Developer Portal: momodeveloper.mtn.com
- Create a product subscription for 'Collections'
- Obtain: Primary Key (subscription key), Secondary Key (backup)

Step 2: Sandbox Testing

- Use the MTN MoMo Sandbox environment for all development and QA
- Sandbox base URL: https://sandbox.momodeveloper.mtn.com
- Create a sandbox API User via POST /v1_0/apiuser
- Generate API Key via POST /v1_0/apiuser/{X-Reference-Id}/apikey
- Obtain Bearer token via POST /collection/token/ with Basic auth

Step 3: Production Onboarding

- Submit production access request to MTN Rwanda (include business registration, use case description)
- MTN Rwanda reviews and provisions production credentials
- Production base URL: https://proxy.momoapi.mtn.com
- Store all production credentials in environment variables — never hardcode

8.3 Collections API Integration Code Pattern

Key API Calls

Action	Endpoint	Method	Notes
Request to Pay (initiate collection)	/collection/v1_0/requesttopay	POST	Sends USSD prompt to customer's phone
Check payment status	/collection/v1_0/requesttopay/{referenceId}	GET	Poll to verify payment
Get account balance	/collection/v1_0/account/balance	GET	Check MoMo balance
Validate account holder	/collection/v1_0/accountholder/msisdn/{phone}/active	GET	Verify phone is active MoMo user

Request to Pay — Required Payload

- amount: string — the amount in RWF (e.g. '15000')
- currency: 'RWF'
- externalId: string — your internal invoice ID (for reconciliation)
- payer.partyIdType: 'MSISDN'
- payer.partyId: string — customer's MTN phone number (e.g. '0781234567')
- payerMessage: string — message shown to customer on their phone (e.g. 'StackFix Invoice INV-0042')
- payeeNote: string — your internal reference note
- Required headers: X-Reference-Id (your UUID), X-Target-Environment, Ocp-Apim-Subscription-Key, Authorization: Bearer {token}

Callback Webhook Handler

MTN will POST to your callback URL when payment status changes. The handler must:

48. Immediately return HTTP 200 — MTN retries if it doesn't get a fast 200
49. Verify the callback signature (X-Callback-Signature header)
50. Enqueue a background job to process the payment status asynchronously
51. Background job: fetch payment status from /requesttopay/{referenceId}
52. If status is SUCCESSFUL: mark invoice paid, trigger ticket status update, send confirmation notifications
53. If status is FAILED: log, do not update invoice, optionally notify technician

8.4 Payment Reconciliation

Never rely solely on webhooks — build a reconciliation job:

- BullMQ job runs every 5 minutes
- Fetches all invoices with status 'sent' and payment records with status 'pending'
- For each pending payment, calls GET /collection/v1_0/requesttopay/{referenceId}
- Updates payment status based on MoMo's response
- Handles cases where webhook was missed or arrived out of order

8.5 USSD Code Generation

StackFix generates the USSD string for each invoice. The format for MTN MoMo Rwanda:

- Format: *182*8*1*{AMOUNT}*{BUSINESS_CODE}# (confirm exact format with MTN Rwanda)
- This code is embedded in the invoice PDF, WhatsApp message, and SMS
- The reference/external ID is stored on the invoice so incoming webhooks can be matched

8.6 Airtel Money (Future Integration)

A significant portion of Rwanda uses Airtel Money. Plan for this from the start:

- Database payment_method ENUM already includes 'airtel_money' as a value
- Payment service layer is abstracted — adding Airtel Money is a new provider implementation behind the same interface
- Airtel Rwanda Business API: developer portal documentation available on request
- Launch without Airtel — add in Phase 2 post-launch based on customer demand

8.7 Financial Compliance in Rwanda

- All payment records retain for minimum 7 years (Rwanda Revenue Authority requirement)

- VAT invoices at 18% rate as required by RRA — StackFix calculates and displays VAT separately
- EBM (Electronic Billing Machine) compliance: work with local accountants to determine if StackFix needs to integrate with RRA's EBM system for VAT reporting
- No financial data is ever purged — soft deletes only, full audit trail maintained

PHASE 9

WhatsApp & SMS Notification System

Automated customer notifications are a key StackFix value proposition. They eliminate the follow-up phone call and build customer trust. This system must be reliable, fast, and bilingual.

9.1 Notification Strategy

Every notification is sent on a trigger event. Default channel order: WhatsApp first (preferred), SMS as fallback if WhatsApp fails. Admins can configure per-channel preferences.

9.2 Notification Trigger Matrix

Event	Trigger	Message Type	Channel
Ticket created	System auto	Device received confirmation	WhatsApp + SMS
Invoice sent	System auto	Invoice + payment instructions	WhatsApp + SMS
Payment confirmed (MoMo)	System auto	Payment received + repair start	WhatsApp + SMS
Repair completed	Technician marks complete	Device ready for pickup	WhatsApp + SMS
Pickup model — ready	Technician marks complete	Ready + invoice + USSD code	WhatsApp + SMS
Ticket cancelled	Admin action	Cancellation notice	WhatsApp + SMS
Reminder (optional)	Scheduled job (24h/48h)	Uncollected device reminder	WhatsApp + SMS

9.3 WhatsApp Business API Integration

Setup

- Create a Meta Business account at business.facebook.com
- Create a WhatsApp Business Account (WABA) within Meta Business Suite

- Register a dedicated business phone number for StackFix (cannot be personal WhatsApp)
- Apply for WhatsApp Business API access — use a Meta-approved BSP (Business Solution Provider) or direct API
- Recommended BSP for Africa: Twilio, Vonage, or Africa's Talking (all support WhatsApp + fallback SMS in one platform)
- Obtain: Access Token, WhatsApp Business Account ID, Phone Number ID

Message Templates (Required for Outbound Messaging)

WhatsApp requires pre-approved message templates for business-initiated messages. Submit all templates to Meta for approval before launch. Approval typically takes 24–72 hours.

Required templates to create and submit:

Template Name	Language	Variables	Sample Message
device_received	EN + RW	{{customer_name}}, {{ticket_id}}, {{device}}	Hi {{1}}, your {{2}} (ticket #{{3}}) has been received. We'll keep you updated.
invoice_sent	EN + RW	{{customer_name}}, {{amount}}, {{ussd_code}}	Your invoice for RWF {{2}} is ready. Pay now by dialling {{3}} on your phone.
payment_confirmed	EN + RW	{{customer_name}}, {{ticket_id}}	Payment confirmed! Your device (ticket #{{2}}) is now being repaired.
repair_complete	EN + RW	{{customer_name}}, {{device}}	Great news! Your {{2}} is repaired and ready for pickup. Come collect it at your convenience.
pickup_invoice	EN + RW	{{customer_name}}, {{device}}, {{amount}}, {{ussd_code}}	Your {{2}} is ready! Please pay RWF {{3}} by dialling {{4}}, then come collect.
device_reminder	EN + RW	{{customer_name}}, {{device}}, {{days}}	Reminder: your {{2}} has been ready for {{3}} days. Please collect it soon.

Sending Messages via API

Use the WhatsApp Cloud API (free via Meta direct) or BSP SDK:

- Endpoint: POST https://graph.facebook.com/v18.0/{phone_number_id}/messages
- Header: Authorization: Bearer {WHATSAPP_API_TOKEN}
- Body: { messaging_product: 'whatsapp', to: '{customer_phone}', type: 'template', template: { name: '...', language: { code: 'en' }, components: [...] } }

Delivery Status Webhook

- Configure Webhook URL in Meta Developer Console to receive delivery status callbacks
- Statuses: sent → delivered → read — track all three in notifications table
- Failed delivery triggers automatic SMS fallback via BullMQ job

9.4 SMS Integration

Provider: Africa's Talking

Africa's Talking is the leading SMS API provider in East Africa with direct carrier connections in Rwanda:

- Register at africastalking.com — create account and application
- Request a shortcode or sender ID registration in Rwanda (e.g. 'STACKFIX') — submit to Africa's Talking for carrier approval
- Sender ID approval takes 1–2 weeks — start this process early
- Obtain: API Key, Username
- Rwanda SMS pricing: approximately USD 0.02–0.04 per message (cheaper in bulk)

SMS Sending via Africa's Talking SDK

- Install: npm install africastalking
- Initialise with apiKey and username
- Call AT.SMS.send({ to: ['+250781234567'], message: '...', from: 'STACKFIX' })
- Handle delivery receipts via Africa's Talking webhook (configure in AT dashboard)

SMS Message Format

SMS messages must be short and contain the essential information:

- Max 160 characters for single-part SMS (longer messages are concatenated and charged per 153-char segment)

- Always include ticket number, key status, and action required
- USSD code must be included in payment-related SMS
- Example: 'StackFix: Your Samsung A54 (TKT-0042) is ready. Pay RWF 15,000 by dialling *182*8*1*15000*42# then come collect. -StackFix'

9.5 Notification Service Architecture

Service Layer Pattern

54. Business logic triggers notificationService.send(event, ticketId)
55. Service builds message payload from template + ticket data
56. Service enqueues job in BullMQ 'notifications' queue
57. WhatsApp worker picks up job, sends via Meta API
58. If WhatsApp fails (non-200 response or delivery failure callback), SMS worker is triggered
59. Both delivery outcomes logged to notifications table
60. Failed notifications after 3 retries: alert admin via dashboard notification

9.6 Bilingual Notification Logic

- Customer notification language determined by: organisation's default language setting
- Future enhancement: customer language preference stored on customer record
- WhatsApp templates must be approved in both 'en' and 'rw' language codes
- SMS messages stored in i18n format alongside web app translations
- All Kinyarwanda translations reviewed by a native speaker before production

PHASE 10

Mobile App Development

The StackFix mobile app gives technicians a native experience and allows shop owners to monitor their business on the go. It shares the same API as the web app.

10.1 Mobile App Strategy

Platform: React Native via Expo SDK. This choice is made for three reasons:

61. Code sharing with the web app — same TypeScript logic, same API calls, same design tokens
62. Single codebase for both iOS and Android — critical for a small team
63. Expo EAS (Expo Application Services) provides professional build and OTA update infrastructure without managing native toolchains

10.2 Target Devices

Rwanda's smartphone market is dominated by:

- Android (80%+): Samsung Galaxy A-series, Tecno, Infinix, itel — prioritise Android
- iOS (20%): iPhone SE, iPhone 12–14 (among higher-income users)
- Minimum Android API level: 24 (Android 7.0 Nougat)
- Minimum iOS: 14

10.3 Mobile Tech Stack

Technology	Purpose	Notes
Expo SDK 51+	React Native framework and tooling	Managed workflow for simplicity
React Native 0.74+	Cross-platform UI	
TypeScript	Type safety	Shared types from packages/types
Expo Router	File-based routing	Same pattern as Next.js App Router
React Native Paper	Material Design UI components	Extended with StackFix tokens

TanStack Query	API data fetching	Same library as web app
Zustand	Client state	Same library as web app
Expo Notifications	Push notifications	FCM (Android) + APNs (iOS)
Expo SecureStore	Secure credential storage	Replace localStorage — encrypted
Expo Camera	Barcode/QR scanning (future)	Device IMEI scanning
React Native MMKV	Fast local storage	Cache and offline support
Sentry	Error tracking	Expo Sentry plugin
react-i18next	Internationalisation	Same translations as web app

10.4 Mobile Screen Inventory

Authentication

- Login screen — email + password, biometric login toggle
- Forgot password — email entry, check email screen

Dashboard (Technician view — simplified)

- My active tickets — list with status badges, quick status update swipe actions
- Stats bar — today's completions, pending count
- Quick action: New Ticket button

Dashboard (Admin view)

- KPI cards — total tickets, revenue today, pending count
- Recent activity list
- All technicians' active jobs

Ticket Screens

- Ticket list — same filters as web, pull-to-refresh

- Ticket detail — full info, status update button, invoice link, notes
- New ticket form — customer info, device, fault, AI suggest

Invoice Screens

- Invoice list — filter by status
- Invoice preview — PDF viewer
- Share invoice — WhatsApp deep link to send invoice to customer

Settings

- Profile settings — name, password change
- Notification preferences
- Language toggle
- Log out

10.5 Push Notifications (Mobile)

In addition to WhatsApp/SMS for customers, the mobile app receives push notifications for the technician:

- New ticket assigned to me
- Payment confirmed on my ticket
- Admin message or status update

Infrastructure:

- Expo Notifications handles FCM (Android) and APNs (iOS) token registration
- Device push tokens stored in user_push_tokens table in database
- Backend sends push via Expo Push API: POST https://exp.host/--/api/v2/push/send
- Deliver push payload alongside WhatsApp/SMS in the same BullMQ notification job

10.6 Mobile Build & Release Process

Development Build

- expo start — local dev server with hot reload
- Expo Go app for initial testing on physical devices
- Development build (expo run:android / expo run:ios) for native module testing

Production Build (EAS Build)

- eas build --platform android --profile production → generates signed .aab for Play Store
- eas build --platform ios --profile production → generates .ipa for App Store
- Signing credentials managed by EAS — no local keystore management needed

App Store Submission

- Google Play Store: register StackForgeAI developer account ($25 one-time fee). Submit .aab, fill store listing, set pricing (free app, paid subscription managed server-side).
- Apple App Store: register Apple Developer Program ($99/year). Submit via App Store Connect. Apple review takes 1–3 days.
- App Store descriptions in English; plan Kinyarwanda store listing for future

Over-The-Air Updates (OTA)

- Non-native code changes (JavaScript, assets) deployed via eas update without app store resubmission
- EAS Update channel per environment: preview → production
- OTA updates available to users within seconds of deployment

PHASE 11

Testing & Quality Assurance

Quality is non-negotiable. A payment bug or a missed notification costs a real business real money. The testing strategy covers every layer of the application.

11.1 Testing Philosophy

The testing pyramid for StackFix:

- Many unit tests — fast, cheap, test pure functions and business logic
- Some integration tests — test API endpoints with real database
- Fewer E2E tests — test complete user journeys in a real browser
- Targeted MoMo payment tests — simulate sandbox payments end-to-end

11.2 Unit Testing

Area	Tool	Coverage Target
Business logic (status transitions, invoice calculations)	Vitest	≥ 90%
Utility functions (USSD code generation, formatting)	Vitest	100%
API route handlers (mocked dependencies)	Vitest + Supertest	≥ 80%
React components (interaction + rendering)	Vitest + Testing Library	≥ 70%
Zod validation schemas	Vitest	100% of all edge cases
Notification message building	Vitest	≥ 90%

11.3 Integration Testing

Integration tests run against a real test PostgreSQL database (Docker container in CI):

- Test full API request → DB → response cycles
- Test ticket status transition sequences end-to-end
- Test invoice creation, line item calculation, PDF generation
- Test MoMo webhook handler with simulated payloads from MTN Sandbox
- Test WhatsApp notification trigger chain
- Test multi-tenant isolation — organisation A cannot access organisation B's data

11.4 End-to-End Testing

Tool: Playwright — test real browser interactions against staging environment:

- New ticket creation flow — complete form, submit, verify ticket appears in list
- Invoice creation and PDF download
- Status change workflow — Pending → Under Repair → Completed → Picked Up
- Admin team management — add/remove technician
- Export — CSV download and verify column structure
- Language switch — toggle EN/RW, verify key strings change
- Login/logout flow and role-based access verification

11.5 Mobile App Testing

- Unit tests for all business logic (same Vitest config shared with web)
- Detox — native E2E testing for React Native (iOS + Android)
- Manual QA on physical devices: Samsung Galaxy A54, Tecno Spark, iPhone SE
- Test on real MTN SIM with sandbox credentials for USSD flow
- Test push notifications end-to-end on Android and iOS

11.6 MoMo Payment Testing Protocol

This requires its own testing protocol due to the real-money implications:

64. Use MTN MoMo Sandbox exclusively during development — never test with real money
65. Sandbox provides test MSISDN numbers that simulate successful and failed payments
66. Test all payment states: SUCCESSFUL, FAILED, PENDING (timeout)
67. Test webhook delivery: verify StackFix processes the callback within 5 seconds
68. Test reconciliation job: manually delay callback, verify job picks up the payment
69. Test duplicate webhook prevention: send same callback twice, verify payment not doubled

70. Production go-live: process one real test payment before announcing to customers

11.7 Performance Testing

- Load test API with k6: simulate 100 concurrent users running the full repair workflow
- Target: P95 response time < 200ms for all read endpoints, < 500ms for write endpoints
- Test dashboard query performance with 10,000 tickets in the database
- Test PDF generation under load — Puppeteer has concurrency limits

11.8 Security Testing

- Run OWASP ZAP against staging API — check for common vulnerabilities
- Manual penetration test before launch: SQL injection, XSS, CSRF, IDOR
- Test JWT token expiry and refresh logic
- Test org isolation: attempt to access another org's tickets with valid token — must return 403
- Test rate limiting: exceed rate limits and verify 429 responses

11.9 QA Sign-off Checklist

No feature ships to production without sign-off on:

- All automated tests pass in CI
- Playwright E2E tests pass against staging
- Visual regression check in Chromatic (no unintentional UI changes)
- Manual QA review by a team member who did not write the code
- Mobile app tested on Android and iOS physical devices
- Performance benchmarks within targets
- WCAG accessibility check passes

PHASE 12

Hosting, DevOps & Infrastructure

StackFix's infrastructure must be reliable, cost-efficient, scalable from 10 to 10,000 repair shops, and maintainable by a small team without dedicated DevOps engineers.

12.1 Infrastructure Philosophy

Managed services over self-managed infrastructure. Every infrastructure decision favours developer productivity and operational simplicity over maximum control. The team should be building product, not managing servers.

12.2 Hosting Stack

Service	Provider	Purpose	Tier to Start
Frontend (Next.js)	Vercel	Web app hosting, CDN, edge functions, CI/CD	Vercel Pro ($20/mo)
API (Node.js)	Railway	Containerised backend, managed PostgreSQL option, Redis	Railway Starter ($5/mo)
Database	Supabase	PostgreSQL, Auth, Realtime, Storage	Supabase Pro ($25/mo)
Cache / Queues	Upstash	Serverless Redis (BullMQ compatible)	Pay per request
File Storage	Supabase Storage	Invoice PDFs, business logos	Included in Supabase Pro
Media / Assets	Cloudflare R2	Static assets if CDN needed (alt to Supabase)	Free up to 10GB
Email	Resend	Transactional emails (password reset, reports)	Free tier 3,000/mo
Error Tracking	Sentry	Frontend + backend error monitoring	Free tier 5,000 errors/mo
Logging	Logtail (BetterStack)	Structured log aggregation	Free tier

Uptime Monitoring	BetterStack Uptime	Downtime alerts via SMS/email	Free tier
DNS	Cloudflare	DNS management + DDoS protection	Free
Domain	stackfix.rw	Primary domain	Register via Ricta (Rwanda domain registry)

12.3 Environment Strategy

Environment	URL	Purpose
Development	localhost:3000 (web), localhost:4000 (api)	Local developer machines — Docker for DB/Redis
Staging	staging.stackfix.rw	Pre-production — auto-deployed from develop branch
Production	app.stackfix.rw	Live customer environment — deployed from main with approval gate

12.4 Domain Structure

- stackfix.rw — landing page / marketing site
- app.stackfix.rw — web application
- api.stackfix.rw — REST API
- staging.stackfix.rw — staging app
- staging-api.stackfix.rw — staging API
- docs.stackfix.rw — API documentation (Swagger UI)

12.5 Vercel Configuration (Frontend)

- Connect Vercel to GitHub repo — auto-deploy on merge to main
- Preview deployments for every PR — share URL with designers for review
- Configure environment variables in Vercel dashboard
- Edge network ensures fast loading from both Rwanda and diaspora markets
- Custom domain: app.stackfix.rw with automatic SSL

12.6 Railway Configuration (API)

- Deploy Node.js API as a Docker container (provide Dockerfile)
- Railway manages container scaling, restarts, and SSL
- Configure environment variables in Railway dashboard
- Set up health check endpoint: GET /health → returns { status: 'ok', version: '...' }
- Railway deploys new version with zero-downtime rolling update
- Set minimum 2 replicas in production for high availability

12.7 Supabase Configuration

- Enable Point-in-Time Recovery (PITR) — 7-day restore window
- Enable Supabase Auth only if using it for admin authentication (optional — can use custom JWT)
- Enable Realtime on the tickets table for live dashboard updates
- Configure Row Level Security policies for all organisation-scoped tables
- Set up Storage buckets: invoices-pdf (private), organisation-logos (public)
- Configure database connection pooling via Supabase's built-in PgBouncer

12.8 Security & Compliance

- All traffic over HTTPS/TLS — enforced at Vercel and Railway
- Cloudflare proxy in front of both domains — DDoS protection
- Database backups: Supabase automated daily backups + manual backup before every major deployment
- Secrets management: all production secrets in Vercel/Railway env vars — never in code
- Access to production database restricted to VPN or specific IPs only
- GDPR-adjacent data practices: customer data stored in Supabase (EU Frankfurt region) or request Africa region when available

12.9 Scaling Path

When the platform grows beyond initial capacity:

- Vertical scale API: upgrade Railway instance size — no code changes needed
- Horizontal scale API: increase Railway replica count — stateless API handles this automatically
- Database scaling: Supabase read replicas for read-heavy analytics queries
- Redis scaling: Upstash scales automatically — no action needed
- CDN scaling: Vercel Edge Network scales automatically

- Migrate to AWS Africa (Cape Town) when African region pricing and support is more favourable

PHASE 13

Launch Strategy

A great product launched poorly fails. A focused, staged launch lets StackFix gather real feedback quickly while protecting reputation.

13.1 Pre-Launch Checklist

Technical

- All Phase 1–12 items completed and signed off
- Production environment fully configured and tested
- MTN MoMo production credentials live and tested with real transaction
- WhatsApp Business API approved and message templates live
- SMS sender ID 'STACKFIX' approved by Africa's Talking + MTN Rwanda
- SSL certificates valid on all domains
- Uptime monitoring alerts configured (alert to team WhatsApp group)
- Sentry error tracking live and alerting
- Database backups verified and tested (restore rehearsal completed)
- Privacy Policy and Terms of Service published at stackfix.rw/legal

Business

- RDB (Rwanda Development Board) business registration complete
- MTN MoMo for Business account active
- Bank account linked to MoMo business for settlements
- Pricing plans finalised: Starter (RWF 29,000/mo), Growth (RWF 89,000/mo), Enterprise (Custom)
- Support email configured: support@stackfix.rw
- Onboarding guide published (PDF + video) in English and Kinyarwanda

13.2 Beta Launch (Weeks 1–4 Post-Build)

Target: 5 repair shops in Kigali who are known contacts or early believers:

71. Personally onboard each beta shop — sit with the technician on day one
72. Free subscription for 3 months in exchange for weekly structured feedback
73. WhatsApp group with all beta shops for direct communication
74. Daily check-ins during week 1, weekly thereafter
75. Collect NPS score weekly, document every bug and friction point
76. Fix critical bugs within 24 hours, improvements within 1 week

13.3 Public Launch

After 4 weeks of successful beta with no critical issues:

- Launch blog post + demo video — distribute via local tech communities (Klab, Rwanda ICT Chamber)
- Product Hunt submission — announce to global audience
- Partner with MTN Rwanda on a co-marketing announcement (leverage MoMo integration)
- Reach out to repair shop associations and phone distributor networks in Rwanda
- Offer a 30-day free trial for all new sign-ups at launch

13.4 Onboarding Flow

The sign-up to first ticket created in under 10 minutes:

77. Shop owner visits stackfix.rw → clicks 'Start Free Trial'
78. Email + business name entry — verify email
79. Quick setup wizard: business details (name, phone, address, logo)
80. Payment model selection (Pay Before / Pay on Pickup)
81. Add first technician (optional — can skip and do later)
82. Tutorial overlay on first dashboard visit — guided tour of key features
83. Prompted to create first test ticket — completes the activation loop
84. Welcome WhatsApp message from StackFix support sent to shop owner's phone

PHASE 14

Post-Launch Operations & Scaling

The product is live. Now the work of growing, maintaining, and improving it begins. This phase never ends.

14.1 Monitoring & Observability

What to Monitor	Tool	Alert Threshold
API uptime	BetterStack Uptime	Downtime > 1 min → SMS + email to team
API error rate	Sentry	> 1% error rate → immediate Slack alert
API response time	Railway metrics	P95 > 500ms → investigate
Database connections	Supabase dashboard	Near connection limit → alert
MoMo webhook failures	Custom logging (Logtail)	Any failure → immediate alert
WhatsApp delivery rate	Notifications table	Delivery rate < 90% → alert
Failed payment records	Custom query	Daily report to team
New sign-ups	Analytics	Daily digest to team

14.2 Customer Support Operations

- Primary channel: WhatsApp Business (dedicate a number for support)
- Secondary: support@stackfix.rw email
- Response time SLA: < 4 hours during business hours (8am–6pm CAT)
- Track issues in a shared Notion or Linear board
- Common issues documented in knowledge base (Notion public page)
- Video tutorials for top 5 user actions — hosted on YouTube and embedded in app

14.3 Deployment Cadence

Release Type	Frequency	Process

Bug fixes	As needed (same day for critical)	Hotfix branch → main → deploy with approval
Feature releases	Bi-weekly	Feature flag controlled — gradual rollout
Mobile app OTA	Weekly if no native changes	eas update → auto-delivered
Mobile app store release	Monthly or as needed for native changes	Full review cycle
Dependency updates	Monthly	Dependabot PRs reviewed and merged
Security patches	Immediate on discovery	Hotfix process

14.4 Product Roadmap (Post-Launch)

Immediate Priorities (Month 1–3)

- Kinyarwanda full translation (complete 100% of UI strings)
- Airtel Money integration (based on beta feedback frequency)
- Customer portal — customers can view their own repair status by entering phone number
- Bulk SMS campaigns — notify all customers of promotions

Growth Phase (Month 3–6)

- Parts inventory management — track spare parts stock
- Multi-shop support — one admin account manages multiple branches
- EBM integration — Rwanda Revenue Authority electronic billing compliance
- Customer loyalty tracking — repeat customer badges and history
- WhatsApp chatbot — customers can check status by messaging StackFix number

Scale Phase (Month 6–12)

- Expand to Uganda, Kenya — adapt for local payment methods (M-Pesa etc)
- Marketplace — connect shops with spare parts suppliers
- Advanced analytics — demand forecasting, technician productivity AI insights
- White-label option — enterprise customers can brand the platform

14.5 Key Metrics to Track Weekly

Metric	Formula	Target (6 months)	Notes
MRR	Sum of active subscriptions × monthly fee	RWF 5,000,000+	Monthly Recurring Revenue
Active Shops	Shops with ≥ 1 ticket in last 30 days	100+ shops	
Churn Rate	(Cancelled this month ÷ Start of month shops) × 100	< 3%/month	
Tickets Processed	Total tickets created (all shops)	5,000+/month	Proves actual usage
USSD Payments	Payments processed via MoMo	70%+ of all invoices	Core value metric
NPS Score	Net Promoter Score survey monthly	≥ 50	
Support Response Time	Avg time to first response	< 2 hours	
Uptime	API availability	99.9%	

14.6 Team Structure (Recommended)

Role	Headcount	Responsibilities
Full-Stack Engineer	2	Web app, API, database — core product
Mobile Engineer	1	React Native app (can be one of the full-stack engineers)
Designer	1 (part-time or contract)	UI/UX, design system maintenance
Customer Success	1	Onboarding, support, NPS surveys, beta feedback
Founder/Product	1	Roadmap, partnerships, sales, strategy

Appendix: Complete Tool Stack Summary

A.1 Full Tool Reference

Category	Tool / Service	Purpose	Priority
Design	Lovable.app	Design-to-code scaffold	Active
Design	Figma	Design specs + component documentation	High
Design	Storybook	Component library visual documentation	High
Design	Chromatic	Visual regression testing	Medium
Frontend	Next.js 14	React framework	Critical
Frontend	TypeScript	Type safety	Critical
Frontend	Tailwind CSS	Styling	Critical
Frontend	shadcn/ui	Base component library	High
Frontend	TanStack Query	Server state management	Critical
Frontend	Zustand	Client state	High
Frontend	React Hook Form + Zod	Form + validation	High
Frontend	Recharts	Data charts	Medium
Frontend	i18next	EN/RW translations	High
Backend	Node.js 20 LTS	Runtime	Critical
Backend	Express.js	HTTP framework	Critical
Backend	Prisma ORM	Database access layer	Critical
Backend	BullMQ	Background job queues	Critical
Backend	Puppeteer	PDF invoice generation	High

Database	PostgreSQL 15 (Supabase)	Primary data store	Critical
Database	Redis (Upstash)	Cache + job queues	Critical
Mobile	React Native + Expo	iOS + Android app	High
Mobile	Expo Router	Mobile routing	High
Mobile	EAS Build	App builds and OTA updates	High
Payments	MTN MoMo Collections API	USSD payment processing	Critical
Payments	Airtel Money API	Alternative payment (Phase 2)	Planned
Notifications	Meta WhatsApp Cloud API	WhatsApp Business messaging	Critical
Notifications	Africa's Talking SMS	SMS fallback + delivery	Critical
Notifications	Expo Push API	Mobile push notifications	High
Notifications	Resend	Transactional email	Medium
Hosting	Vercel	Frontend deployment + CDN	Critical
Hosting	Railway	API deployment	Critical
Hosting	Supabase	DB + Auth + Storage + Realtime	Critical
Hosting	Cloudflare	DNS + DDoS protection	High
DevOps	GitHub Actions	CI/CD pipelines	Critical
DevOps	Turborepo	Monorepo build orchestration	High
DevOps	Docker	Local dev containers	High
Monitoring	Sentry	Error tracking	High
Monitoring	Logtail (BetterStack)	Structured logging	High
Monitoring	BetterStack Uptime	Uptime monitoring	High
Testing	Vitest	Unit + integration tests	Critical

Testing	Playwright	E2E browser tests	High
Testing	Detox	Mobile E2E tests	Medium
Testing	k6	Load testing	Medium
AI	OpenAI GPT-4o or Anthropic API	AI assist features	Medium
Management	Linear	Issue tracking + sprint planning	High
Management	Notion	Documentation + knowledge base	High
Management	Figma	Design + handoff	High

A.2 Rwanda-Specific Contacts & Resources

Resource	Contact / URL	Purpose
MTN MoMo Developer Portal	momodeveloper.mtn.com	API access + sandbox
MTN Rwanda Business	business.mtn.co.rw	MoMo for Business account
Africa's Talking	africastalking.com	SMS API — Rwanda coverage
Ricta (Domain Registry)	ricta.org.rw	Register .rw domain
RDB (Business Registration)	org.rdb.rw	Rwanda Development Board registration
Rwanda Revenue Authority	rra.gov.rw	VAT registration + EBM compliance
Meta WhatsApp Business	business.whatsapp.com	WhatsApp Business API application
RURA (Telecoms Regulator)	rura.rw	SMS sender ID regulatory compliance

StackFix Product Development SOP — v1.0

StackForgeAI · hello@stackforgeai.africa · Kigali, Rwanda · 2026

This document is confidential and intended for internal use only.